MANUAL CREACIÓN DE SERVIDOR CON .NET (C#)

Sebastián Arturo Sosa Gómez

Corporación Unificada Nacional de Educación Superior (CUN)

Escuela de Ingenierías

30428: Ingeniería Web

Docente: David Alberto Ortega Cadena

5 marzo de 2026

Corporación Unificada Nacional de Educación Superior

Dedicatoria

El presente trabajo lo dedico a mi familia, es especial a mis padres, a las normasapa.com, y a mis profesores quienes sin duda han sido una pieza clave y fundamental para mí desarrollo profesional, personal y ético a la hora de desarrollar el presente escrito.

Introducción

.NET es una plataforma de desarrollo de software multiplataforma, de código abierto y de propósito general, mantenida por Microsoft y una activa comunidad global de contribuidores. Desde su replanteamiento como .NET Core y su posterior evolución hacia .NET 5 en adelante, la plataforma ha abandonado su dependencia exclusiva del sistema operativo Windows para ejecutarse de manera nativa en Linux y macOS, consolidándose como una opción competitiva en entornos de servidor, nube y contenedores. El lenguaje principal del ecosistema, C#, combina tipado estático fuerte con características modernas como registros inmutables, pattern matching avanzado, programación asíncrona nativa con async/await y LINQ, lo que facilita la escritura de código expresivo, seguro y mantenible.

El framework ASP.NET Core, componente central de .NET para el desarrollo web y de APIs, proporciona un pipeline de solicitudes HTTP altamente configurable, un sistema de inyección de dependencias integrado de fábrica y soporte nativo tanto para el patrón MVC como para las denominadas Minimal APIs, introducidas para simplificar al máximo la definición de endpoints con el mínimo de código posible.

Palabras Clave: *APIs, C#, Framework, .NET, SDK.*

Objetivos

Objetivo General:

Generar una guía práctica y estructurada para la instalación, configuración y puesta en marcha de de un servidor local utilizando el framework .NET con C#, logrando un entorno de desarrollo funcional, eficiente y alineado con las buenas prácticas recomendadas por Microsoft y la comunidad .NET.

Objetivos Específicos:

- Instalar el SDK de .NET en el sistema operativo del desarrollador y verificar su correcta configuración mediante el comando dotnet --version en la terminal del sistema.
- Crear un nuevo proyecto de tipo Web API o aplicación web utilizando la CLI de .NET (dotnet new), comprendiendo la estructura de archivos y carpetas generada por defecto.
- Gestionar las dependencias del proyecto a través de NuGet, el gestor de paquetes oficial de .NET, añadiendo y actualizando librerías mediante los comandos dotnet add package y dotnet restore.
- Ejecutar el servidor local utilizando el comando dotnet run y verificar su disponibilidad a través del puerto por defecto, accediendo a los endpoints definidos desde el navegador o herramientas como Postman.

Corporación Unificada Nacional de Educación Superior

Tabla de Contenido

Dedicatoria 2
Introducción 3
Objetivos 4
Objetivo General: 4
Objetivos Específicos: 4
Lista de Figuras 6
1. Desarrollo (Paso a Paso) 7
1.1 Buscar en Google el Kit de Desarrollo de Software 7
1.2. Agregar .NET al PATH 7
1.3. Crear Proyecto 8
1.4. Estructura Mínima del Proyecto 9
1.5. Configurar el puerto en “Program.cs”: 9
1.6. Abri la terminal 9
1.7. Abrir Puerto Local 10
2. Cuadro Comparativo 10
2.1. Instalación, Configuración, Ventajas y Desventajas 10
2.2. Detalles Técnicos 12
Conclusiones 13
Referencias 14
Vita 15

Corporación Unificada Nacional de Educación Superior

Lista de Figuras

Figura 1.1 *Página WEB SDK .NET*.....7

Figura 1.2 *Variables de Entorno*.....8

Figura 1.3 *Comando en la Terminal*.....8

Figura 1.4 *Estructura de carpetas del proyecto*.....9

Figura 1.5 *Añadiendo el puerto*. 9

Figura 1.6 *Servidor en ejecución* 9

Figura 1.7 *Servidor Activo* 10

1. Desarrollo (Paso a Paso)

Para desarrollar este proyecto es necesario descargar e instalar las siguientes herramientas en la computadora:

1.1 Buscar en Google el Kit de Desarrollo de Software

En la página de Microsoft buscar “https://dotnet.microsoft.com/es-es/download/dotnet/9.0” o la version requerida.

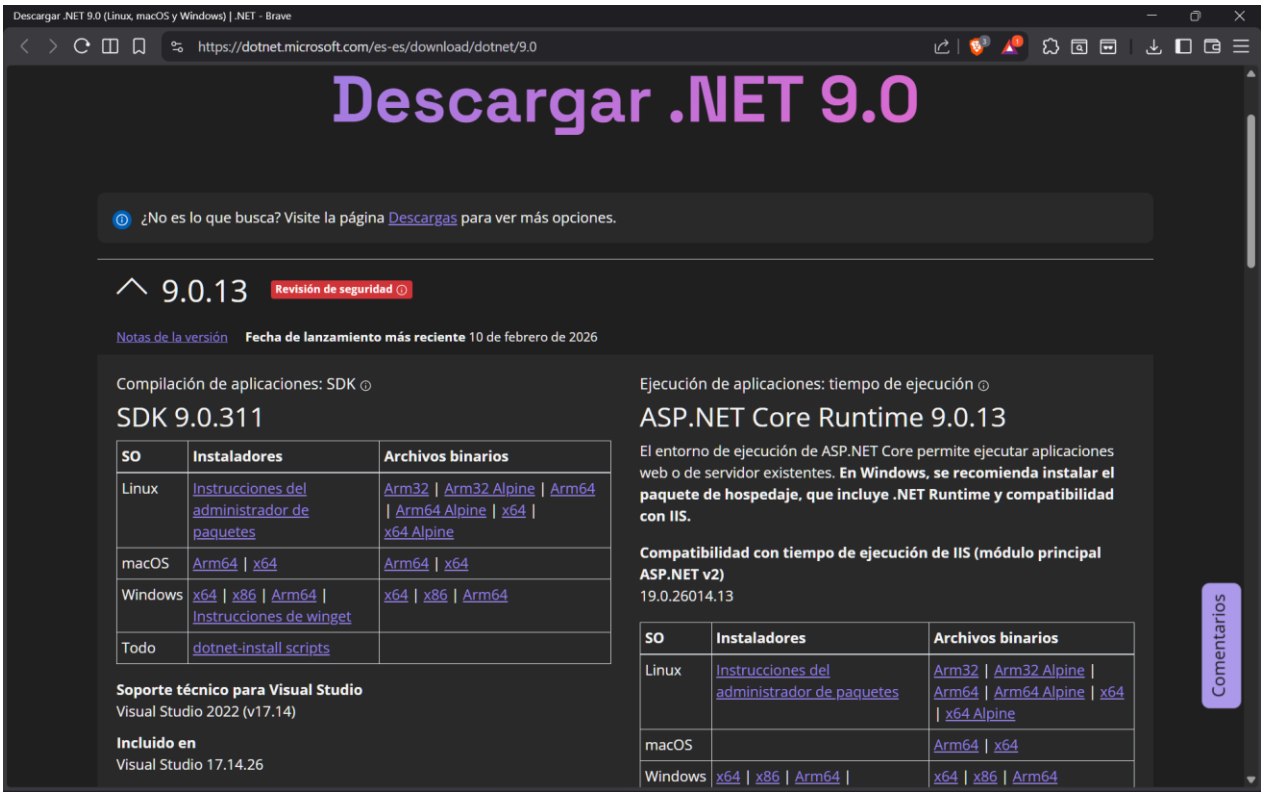

Figura 1.1 Página WEB SDK .NET

1.2. Agregar .NET al PATH

Agregar .NET al path abrir las variables de entorno y buscar “variables del sistema” → Editar → Poner en el path:

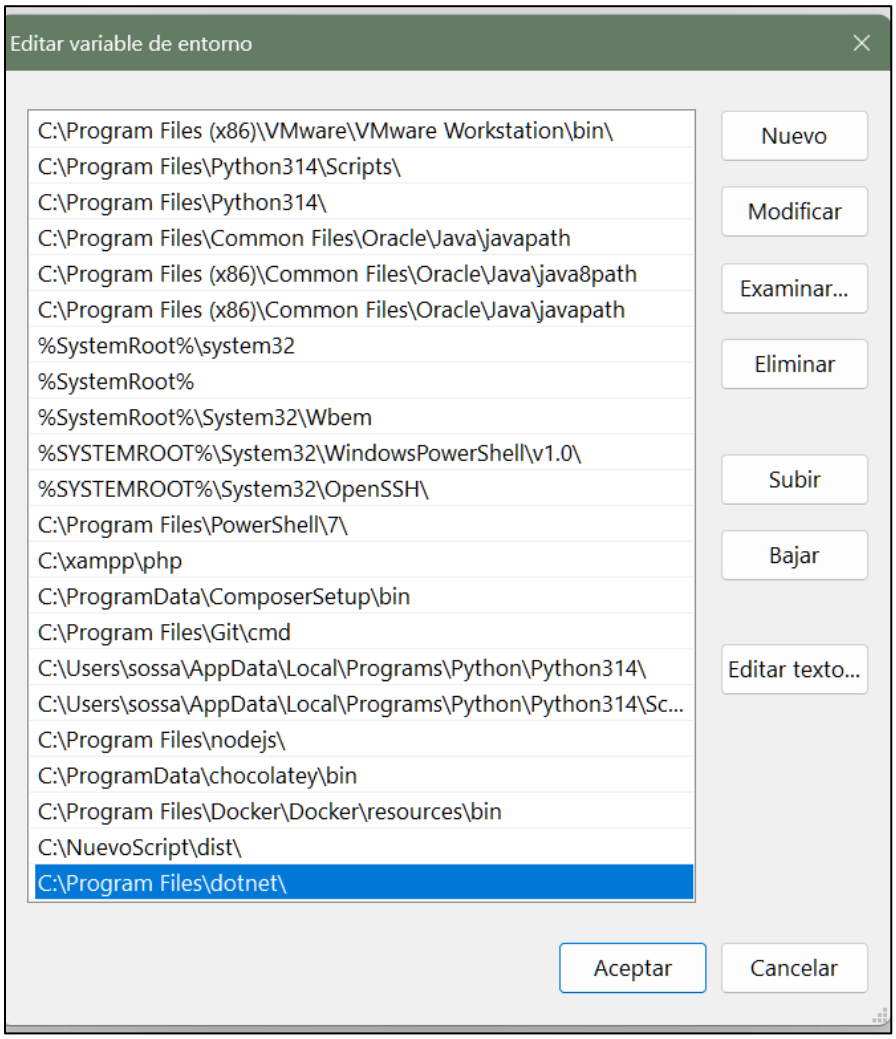

Figura 1.2 Variables de Entorno

1.3. Crear Proyecto

Crear el proyecto usando el siguiente comando en la raíz del proyecto:

Figura 1.3. Comando en la Terminal

1.4. Estructura Mínima del Proyecto

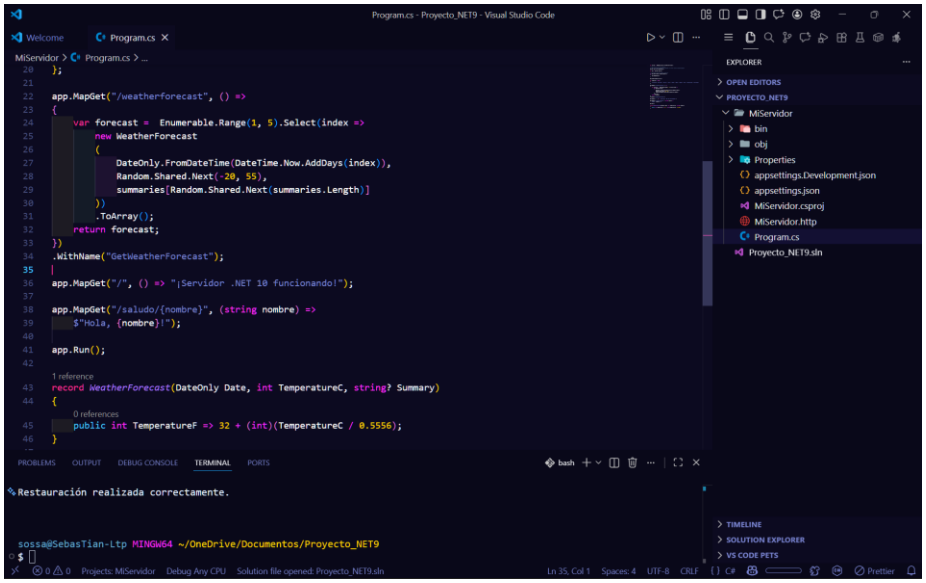

Figura 1.4 Estructura de carpetas del proyecto

1.5. Configurar el puerto en "Program.cs":

Figura 1.5 Añadiendo el puerto

1.6. Abri la terminal

Abrimos la terminal y ejecutamos el comando para levantar el servidor: "dotnet run"

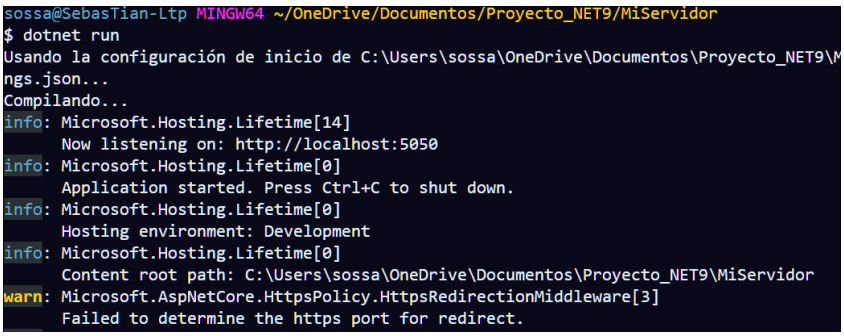

Figura 1.6 Servidor en ejecución

1.7. Abrir Puerto Local

Para verificar que el puerto este activo y escuchando vamos a la dirección:

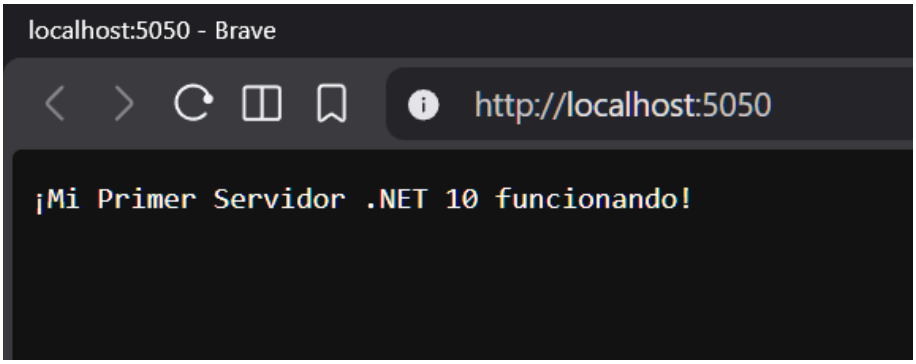

Figura 1.7 Servidor Activo

2. Cuadro Comparativo

A continuación, se va a presentar la comparativa entre distintas tecnologías y frameworks que se utilizan para levantar y configurar un servidor:

2.1. Instalación, Configuración, Ventajas y Desventajas

Parámetros	Node.js	Laravel	Spring Boot JDK 21	.NET (C#)
INSTALACIÓN (DIFICULTAD)	**(Fácil)** Instalar Node.js desde nodejs.org o via nvm. Incluye npm. Sin dependencias externas adicionales.	**(Media)** Requiere PHP ≥8.2, Composer y extensiones PHP. Se recomienda Laragon o Sail (Docker) en Windows.	**(Media)** Instalar JDK 21 + Maven/Gradle. Se puede generar proyecto desde *start.spring.io*. IDE como IntelliJ facilita el proceso.	**(Fácil)** Instalar el SDK .NET desde dotnet.microsoft.com. CLI integrada dotnet. Visual Studio o VS Code con extensión C#.
CONFIGURACIÓN	Archivo package.json + variables de entorno en .env. Sin estructura forzada; se puede usar Express, Fastify, etc.	Archivo .env con claves de BD, caché, mail, etc. Estructura MVC forzada por el framework. Artisan automatiza tareas.	Archivo application.properties o application.yml. Spring Boot Auto-Configuration reduce	appsettings.json / appsettings.{env}.json. Inyección de dependencias nativa. Configuración fuertemente tipada.

			configuración manual. Perfiles por entorno.	
VENTAJAS	• Gran ecosistema npm • E/S no bloqueante (alta concurrencia) • Mismo lenguaje en front y back • Comunidad enorme • Ideal para microservicios y APIs REST	• Desarrollo ágil con convención sobre configuración • Eloquent ORM muy intuitivo • Artisan CLI poderoso • Ecosistema completo (Queues, Broadcasting, etc.) • Documentación excelente	• Alta robustez y madurez empresarial. • Virtual Threads (JDK 21) para concurrencia masiva. • Ecosistema Spring maduro. • Excelente para arquitecturas complejas. • Fuerte tipado y seguridad.	• Alto rendimiento (Minimal APIs, AOT) • Ecosistema Microsoft integrado (Azure, EF Core) • C# moderno y expresivo • Multiplataforma (Linux, macOS, Windows) • Excelente soporte en IDE
DESVENTAJAS	• Callback hell sin control • No ideal para tareas CPU intensivas • Falta de estructura forzada (sin framework) • Versionado rápido puede romper proyectos	• PHP tiene estigma en entornos Enterprise. • Puede ser lento sin OPcache/tuning. • Curva media si no se conoce PHP moderno. • Menos adecuado para microservicios.	• Tiempo de arranque lento (sin GraalVM). • Alta verbosidad y boilerplate. • Curva de aprendizaje pronunciada. • Mayor consumo de memoria.	• Ecosistema más ligado a Microsoft. • Curva media para desarrolladores no-MS. • Algunas librerías open source menos maduras.

2.2. Detalles Técnicos

Parámetros	Node.js	Laravel	Spring Boot JDK 21	.NET (C#)
LENGUAJE BASE	JavaScript (TypeScript opcional)	PHP (≥ 8.2 recomendado)	Java (JDK 21 LTS)	C# (también F#, VB.NET)
GESTOR DE DEPENDENCIAS	npm / yarn / pnpm	Composer	Maven / Gradle	NuGet
LEVANTAR SERVIDOR	node index.js npm run dev (con nodemon o tsx en dev)	php artisan serve	./mvnw spring-boot:run ./gradlew bootRun	dotnet run dotnet watch (watch = hot reload)
DEFINIR UNA RUTA	**Express.js:** app.get('/ruta', (req, res) => res.json({}))	**routes/web.php o api.php:** Route::get('/ruta', [Controller::class, 'método']);	**Anotación en Controller:** @GetMapping("/ruta") public ResponseEntity método() {}	**Minimal API:** app.MapGet("/ruta", () => Results.Ok()); **Controller:** [HttpGet("/ruta")]
PUERTO POR DEFECTO	**3000**	**8000**	**8080**	**5000 / 5001 / 5050**

Conclusiones

La realización de este trabajo, así como su finalización me han permitido deducir las siguientes conclusiones:

Permiten afirmar que .NET representa una plataforma madura, versátil y de alto rendimiento que ha superado con creces su estigma de herramienta exclusiva del ecosistema Microsoft. Su disponibilidad multiplataforma, la potencia expresiva de C# y la integración nativa de herramientas como la CLI de dotnet, el servidor embebido Kestrel y el sistema de inyección de dependencias, permiten levantar un servidor local completamente funcional con una fricción mínima, lo que resulta especialmente valioso en etapas tempranas de un proyecto donde la velocidad de iteración es crítica.

Esta flexibilidad, sumada al soporte de Minimal APIs para escenarios ligeros y de controladores MVC para aplicaciones más estructuradas, hace que .NET se adapte tanto a proyectos pequeños como a sistemas distribuidos de gran escala con iguales garantías de rendimiento y mantenibilidad.

Finalmente, los fundamentos adquiridos constituyen el punto de partida hacia capacidades avanzadas como la contenerización con Docker, el despliegue en Azure App Service, la implementación de arquitecturas de microservicios con .NET Aspire y la integración en pipelines de CI/CD. Dominar estos conceptos base no solo acelera el desarrollo inmediato, sino que sienta las bases técnicas para enfrentar con confianza los desafíos de proyectos de mayor complejidad y exigencia en entornos productivos reales.

Corporación Unificada Nacional de Educación Superior

Referencias

Download Visual Studio Code. (s/f). Visualstudio.com. Recuperado el 5 de marzo de 2026, de https://code.visualstudio.com/download

Download .NET. (s/f). Microsoft. Recuperado el 6 de marzo de 2026, de https://dotnet.microsoft.com/en-us/download

Información general sobre el SDK de .NET - .NET. (s/f). Microsoft.com. Recuperado el 6 de marzo de 2026, de https://learn.microsoft.com/es-es/dotnet/core/sdk

Rivas, A. (2025, marzo 16). *Normas APA con plantilla y generador 2026 - Séptima edición*. Normas APA. https://normasapa.in/

Vita

Soy Sebastián Arturo Sosa Gómez, estudiante de ingeniería de sistemas, quién se encuentra construyendo su propio camino en busca de la gloria. Tengo 24 años, y me apasionan las ciencias, la tecnología, los inventos, la mecánica, la automovilística, veo deportes en especial fútbol, como hobbies me gusta leer de vez en cuando artículos sobre el espacio, noticias tech, geek, avances científicos, noticias sobre videojuegos. Me llama la atención la programación, pero más enfocada en la parte de *reverse coding* o hacking ético, el funcionamiento de las redes, los datos y bases de datos, por ahora esta es mi descripción.